

A Bayesian Information-Gain Agent for CI Failure Diagnosis

Panduru Grishm Siddharth

Independent Researcher

siddharthpg18@gmail.com

Abstract

CI pipeline failures can have several possible root causes, and the available evidence is often insufficient to identify the cause immediately. We developed a CI diagnosis agent that treats the root cause as a hidden state and maintains a probability distribution over possible causes. The V1 agent considers four hidden states: Code, Test, Dependency, and CI/Config. It begins with empirical prior probabilities obtained from a labeled CI failure dataset and updates its beliefs as diagnostic actions produce new evidence. The diagnostic policy ranks actions using expected information gain relative to diagnostic cost (EIG/cost), allowing the agent to prefer actions that are expected to reduce uncertainty while considering diagnostic effort. A 40-case held-out evaluation compared EIG/cost with a cost-only policy and a historical-probability baseline. EIG/cost achieved 55% accuracy, 65.23% macro precision, 55% macro recall, and 84.6% report precision, while also using fewer actions and lower average diagnostic cost than the cost-only policy. However, the experiment uses simulated action outcomes because the original dataset does not directly contain diagnostic-action outcome probabilities. The results therefore demonstrate the behavior of the proposed decision policy under the constructed simulation rather than production-level diagnostic performance.

1 Introduction

CI/CD pipelines can fail for many different reasons. A failure may be caused by a recent code change, a test or flaky test, a dependency problem, or a CI/environment configuration issue. In some cases, the failure can also be related to infrastructure or an external service. The main problem is that the actual cause is not directly observable when the failure first occurs. The agent therefore has to reason about multiple possible causes and decide what to investigate next.

The goal of this project is to build an agent which identifies the cause of a CI pipeline failure and performs diagnostic actions to identify the root cause. Instead of following one fixed debugging sequence, the agent uses the con-

text of the failure to decide which diagnostic action should be performed next. This follows the observation from practitioner discussions that engineers generally start by identifying the failed stage and inspecting logs, and then use the available evidence to narrow down the possible causes and next actions. In one discussion, a practitioner described identifying the failed stage as useful for eliminating possible root causes, while another described using the failed stage, logs, code/tests, dependencies, and infrastructure as part of the debugging process. These observations influenced the design of the agent.

The agent represents the possible root causes as hidden states and maintains a belief, represented as a probability, for each state. Historical data is used to obtain the initial prior distribution. When evidence is observed, the agent updates the probabilities using Bayesian reasoning. The agent can then choose a diagnostic action which is expected to provide useful information about the hidden state.

The central decision problem is therefore not only to predict a root cause, but also to decide which action should be performed next. The proposed V1 policy uses expected information gain divided by diagnostic cost to rank available actions. After an action is performed, its outcome is treated as feedback and used to update the posterior distribution. If a root cause reaches a predefined confidence threshold of 90%, the agent reports it. If the available diagnostic actions are exhausted without reaching this threshold, the agent escalates the case for human review.

The quantitative experiment compares three policies: an EIG/cost policy, a cost-only policy, and a baseline based on historical probabilities. The EIG/cost policy achieved 55% accuracy compared with 50% for cost-only and 25% for the baseline.

The main contributions of this work are:

- A CI failure diagnosis formulation in which the root cause is represented as a hidden state with an associated probability.
- A sequential diagnostic policy that ranks actions according to expected information gain relative to diagnostic cost.
- A Bayesian belief update process that uses diagnostic outcomes as feedback.
- An evaluation of EIG/cost against cost-only and

historical-probability baseline policies on 40 held-out cases.

- A failure analysis identifying cases where the agent became confidently wrong, was diverted by misleading evidence, or remained below the reporting threshold.

2 Related Work and User Discussions

The initial design of the agent was informed by discussions with developers and practitioners about how CI failures are diagnosed in practice. The purpose of these discussions was not to treat individual responses as ground-truth probability data, but to identify useful evidence, diagnostic actions, and situations where human intervention is appropriate.

A response in a Reddit discussion described first exporting and reading CI logs and then attempting to reproduce the failure. The response also noted that identifying the failed pipeline stage can eliminate several possible root causes. This led to treating the failed pipeline stage as an important input to the agent and to including log inspection as a diagnostic action.

Another practitioner response described a debugging sequence beginning with the failed stage and pipeline logs, followed by code/tests, dependencies, and infrastructure/configuration. Previous incidents and runbooks were also described as useful for confirming a diagnosis. These observations support the idea that diagnosis should be sequential rather than based on a single prediction.

A separate practitioner discussion described a build failure workflow in which the engineer first checks the local build and then investigates dependency, configuration, Docker, hosting, and server state. The practitioner also noted that the exact root cause can sometimes remain unknown even after the failure has been localized. This influenced the decision to allow the agent to escalate rather than forcing it to report a root cause when the available evidence is insufficient.

These discussions also influenced the action-selection policy. Rather than using a fixed action sequence for every failure, the research notes propose selecting actions according to the context of the failure and their expected usefulness or “hit ratio.” The final V1 implementation operationalizes this idea using EIG/cost.

The project also received an architecture review in which the high-level Excalidraw architecture was described as clearly explained. The reviewer suggested consolidating the work into a polished LaTeX paper. This influenced the presentation of the final work but was not used as evidence for the quantitative evaluation.

The discussion record is maintained separately as a project artifact and distinguishes direct responses from background observations.

3 Agent Design

The agent receives a CI failure and attempts to identify its root cause through sequential diagnosis. The V1 architecture consists of an evidence collector, a belief engine, a diagnostic policy, diagnostic actions, feedback, and human escalation.

The evidence collector provides the context for the failure. The belief engine maintains probabilities over the possible

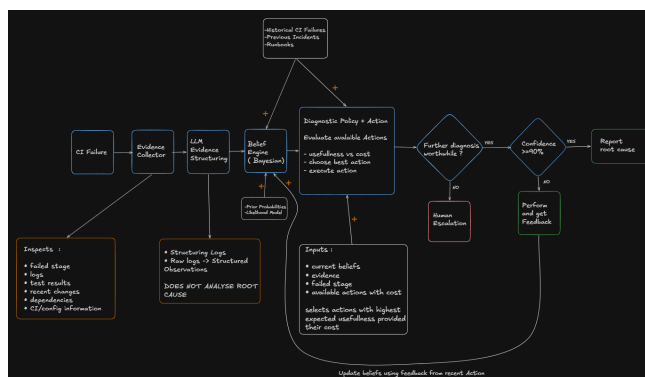


Figure 1: Architecture of the CI Diagnosis Agent. The agent collects evidence, maintains beliefs over hidden root causes, selects diagnostic actions, incorporates action feedback, and escalates to a human when sufficient confidence is not achieved.

root causes. The diagnostic policy decides which available action should be performed. The result of the action is then returned as feedback to the belief engine, which updates the probabilities. If the agent cannot reach sufficient confidence after useful actions have been exhausted, it escalates the case to a human.

3.1 Agent Input

The primary V1 observable evidence variable is the failed pipeline stage. The dataset did not provide a standardized stage field, so stages were derived from pipeline step names using high-confidence mappings. The resulting stages included Build, Test, Setup/Dependency, Quality/Analysis, Deploy/Publish, Ambiguous, and Unknown.

The research also identifies additional evidence that can be used by a richer version of the agent, including pipeline logs, test results, recent code/test changes, dependency changes, CI/environment configuration changes, differences from the last successful run, local build results, Docker image or cache changes, previous similar incidents, hosting information, and server/session state.

However, the V1 quantitative model does not have direct historical observations for all of these variables. The failed stage therefore acts as the first empirically validated evidence variable.

3.2 Hidden States

The V1 quantitative experiment uses four hidden states:

1. **Code:** the failure is caused by a recent code change or application issue.
2. **Test:** the failure is caused by a test-related issue or flaky test.
3. **Dependency:** the failure is caused by a dependency or dependency-related change.
4. **CI/Config:** the failure is caused by a CI, build, or environment configuration issue.

The research also considers Infrastructure and External Service as conceptual root-cause states. However, only two

examples of each were available after mapping the dataset, making them too sparse for reliable quantitative probability estimation in V1. They therefore remain part of the conceptual architecture and are candidates for future expansion.

3.3 Evidence and Diagnostic Actions

The agent can perform several diagnostic actions, including:

- Inspect pipeline logs
- Inspect the failed pipeline stage
- Compare with the last successful run
- Inspect recent code/test changes
- Inspect dependency changes
- Inspect CI/environment configuration
- Search previous incidents/runbooks
- Inspect Docker image/cache changes
- Inspect server/session state

The actions are not intended to be executed in one fixed order. The failed stage and current beliefs are used to determine which actions are relevant. The policy then ranks the available actions using their expected information gain relative to cost.

3.4 LLM Role

A future version of the architecture includes an LLM between the evidence collector and the belief engine. The intended role of the LLM is to structure messy or unstructured CI evidence into a form that the probabilistic model can process.

The LLM is not intended to directly determine the root cause. The research separates evidence interpretation from probabilistic diagnosis: the Bayesian belief engine remains responsible for maintaining the probabilities and the diagnostic policy remains responsible for selecting actions.

4 Probability Model and Decision Rule

The agent represents the unknown root cause as a hidden state. Let

$$H \in \{\text{Code, Test, Dependency, CI/Config}\}.$$

Let E represent the evidence currently available to the agent, a represent a diagnostic action, and o represent an outcome of that action.

4.1 Prior Probability

Before considering evidence from the current case, the agent starts with an empirical prior estimated from the historical labeled dataset. After applying the V1 hidden-state mapping, 305 of the 375 records were retained for quantitative probability estimation.

The resulting empirical prior was:

These probabilities represent the distribution within the included subset of the dataset and are not treated as universal CI failure probabilities.

The prior is conceptually written as:

Table 1: V1 empirical prior over hidden states.

Hidden State	Cases	Prior
Code	54	17.70%
Test	120	39.34%
Dependency	91	29.84%
CI/Config	40	13.11%
Total	305	100%

Table 2: Likelihood of observing a Build failure for each V1 hidden state.

Hidden State	$P(\text{Build} \mid H)$
Code	57.4%
Test	35.8%
Dependency	42.9%
CI/Config	27.5%

$$P(H).$$

The current failure evidence should not be used to construct the prior. It is instead incorporated through the likelihood model.

4.2 Likelihood

The likelihood describes how likely an observation is under a particular hidden state:

$$P(o \mid H, E, a).$$

For the initial V1 evidence model, the failed pipeline stage is represented as a categorical observable variable. For example, the empirical likelihoods for observing a Build failure are:

The likelihood values are empirical estimates from the labeled dataset and are not intended to represent universal CI failure statistics.

4.3 Bayesian Posterior Update

After receiving an observation, the agent updates its belief using Bayes' rule:

$$P(H \mid E, o, a) = \frac{P(o \mid H, E, a)P(H \mid E)}{P(o \mid E, a)}.$$

The denominator normalizes the probabilities:

$$P(o \mid E, a) = \sum_h P(o \mid h, E, a)P(h \mid E).$$

Therefore, the posterior is obtained by multiplying the current belief by the likelihood and then normalizing the result.

4.4 Entropy

The uncertainty in the current belief distribution is represented using entropy:

$$H(H \mid E) = - \sum_h P(h \mid E) \log P(h \mid E).$$

A distribution in which the probabilities are spread across several hidden states has greater uncertainty. A distribution in which one state has most of the probability has lower uncertainty.

4.5 Information Gain

For an action outcome o , the information gain is the reduction in entropy caused by observing that outcome:

$$IG(o) = H(H | E) - H(H | E, o, a).$$

Thus, an outcome is useful when it substantially reduces uncertainty about the hidden root cause.

4.6 Expected Information Gain

Before executing an action, the agent does not know which outcome will occur. Therefore, it calculates the expected information gain across the possible outcomes:

$$EIG(a) = \sum_o P(o | E, a) IG(o).$$

The probability of an outcome is obtained by marginalizing over the possible hidden states:

$$P(o | E, a) = \sum_h P(o | h, E, a) P(h | E).$$

4.7 Action Selection

The diagnostic policy considers both information gain and diagnostic cost. The score of an action is:

$$Score(a) = \frac{EIG(a)}{Cost(a)}.$$

The action with the highest score is selected first.

This means that the policy does not simply choose the action with the highest information gain. An action that provides a large amount of information but requires substantially more diagnostic effort may be ranked below a cheaper action with a better information-to-cost ratio.

4.8 Stopping and Escalation

The reporting threshold for V1 was set to 90%. If any hidden state reaches:

$$P(H | E) \geq 0.90,$$

the agent reports that hidden state as the root cause.

If the available candidate actions are exhausted without any hidden state reaching the threshold, the agent escalates the case for human review.

5 Test Method

5.1 Dataset

The original replication dataset contains 375 labeled CI failure jobs. It contains root-cause descriptions, sub-categories, broader categories, pipeline steps, and associated actions/scripts.

The V1 agent uses a strict hidden-state mapping. Only categories that can be mapped clearly to the four quantitative

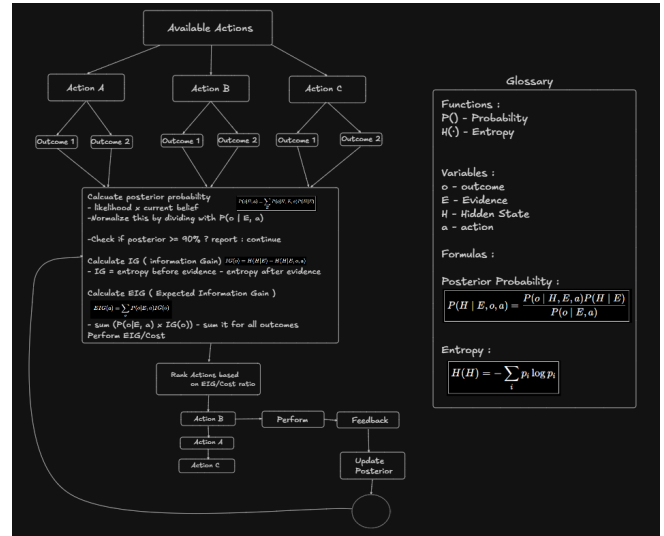


Figure 2: Bayesian and EIG-based action-selection process. The agent maintains a posterior belief, evaluates possible action outcomes, computes information gain and expected information gain, ranks actions by EIG/cost, observes the selected action's outcome, and updates its posterior.

hidden states are included. This resulted in 305 usable records out of the original 375.

Infrastructure and External Service were not used as quantitative V1 states because each had only two mapped examples and was therefore too sparse for reliable probability estimation.

5.2 Evaluation Cases

The agent was evaluated on 40 cases derived from the original dataset. These 40 cases were held out from the data used to construct the agent's priors and outcome models.

The evaluation therefore separates the historical data used to construct the model from the cases used to test the policy.

5.3 Action Outcome Simulation

The original dataset does not directly contain diagnostic-action outcomes. It therefore does not directly provide the probabilities

$$P(o | H, E, a)$$

required for the full EIG calculation.

For V1, these probabilities were estimated using action-specific observable proxies derived from historical step and sub-category fields. The resulting outcome models were used to simulate diagnostic outcomes for the evaluation cases.

This distinction is important: the outcome probabilities are simulation assumptions and are not claimed to be directly observed action probabilities.

5.4 Policies Compared

Three policies were compared.

Baseline. The baseline uses historical probabilities and does not perform diagnostic actions or use a feedback loop.

Cost-only. The cost-only policy ranks available actions by diagnostic cost and performs lower-cost actions first. The motivation is that low-cost actions require less diagnostic effort and may still provide useful information.

EIG/cost. The EIG/cost policy calculates the expected information gain for each available action and ranks actions according to:

$$Score(a) = \frac{EIG(a)}{Cost(a)}.$$

After each action, the observed outcome is incorporated into the posterior distribution and the remaining actions are re-evaluated.

5.5 Evaluation Metrics

The policies were evaluated using:

- Accuracy
- Macro precision
- Macro recall
- Human-review rate
- Average diagnostic cost
- Average number of actions
- Report precision

6 Results

The EIG/cost policy achieved the highest accuracy, macro precision, macro recall, and report precision among the three evaluated policies.

Compared with the cost-only policy, EIG/cost improved accuracy by 5 percentage points and macro recall by 5 percentage points. Macro precision increased by 2.17 percentage points. The human-review rate decreased by 2.5 percentage points. The average diagnostic cost decreased from 6.93 to 6.46, while the average number of actions decreased from 5.23 to 4.68.

The baseline had zero diagnostic cost and zero actions because it does not perform diagnostic actions. However, its accuracy and precision were substantially lower than those of the two sequential diagnostic policies.

The results suggest that considering expected information gain together with diagnostic cost can improve the trade-off between diagnostic effort and root-cause identification under the constructed simulation environment.

7 Failure Analysis

Five failure cases were recorded across three major patterns.

7.1 Confident but Incorrect Diagnosis

In Case 1, the agent became confident in Dependency even though the true root cause was CI/Config. This is an expensive type of failure because the agent performed diagnostic actions and eventually produced a confident but incorrect report.

The research attributes this behavior in V1 partly to the construction of the outcome models. The original dataset

does not contain direct information about action outcomes conditioned on the hidden root cause. The action outcome models were therefore simulated using manually mapped proxies. An incorrect or overly strong simulated outcome can push the posterior towards the wrong hidden state.

7.2 Misleading Evidence

Cases 3, 4, and 10 illustrate failures in which early evidence shifted the posterior toward an incorrect hidden state. Later evidence moved the posterior in the correct direction, but the probability assigned to the incorrect state had already become sufficiently large that the true root cause could not recover enough probability to reach the reporting threshold.

This behavior highlights a limitation of the current outcome model. There were not enough directly observed evidence-outcome relationships to ensure that every simulated piece of evidence would provide a realistic signal for the correct hidden state.

7.3 Correct Direction but Insufficient Confidence

Case 9 reached 88.3% probability for CI/Config but remained below the 90% reporting threshold. In this situation, the diagnostic evidence was moving the belief towards the correct root cause, but the stopping criterion prevented the agent from reporting it.

The 90% threshold was selected heuristically rather than estimated from historical decision outcomes. A larger dataset containing real diagnosis outcomes could be used to calibrate this threshold.

7.4 Code Recall

Both EIG/cost and cost-only achieved 0% recall for Code on the evaluation set. This indicates that the current diagnostic outcome models did not adequately distinguish Code failures from the other root causes in this evaluation.

Overall, the failures can be summarized into three patterns:

1. Confident but incorrect diagnosis.
2. Misleading evidence suppressing the true root cause.
3. Correct evidence but insufficient confidence, resulting in escalation.

8 Limitations, Ethics, and Human Control

The main limitation of the V1 system is the lack of directly observed diagnostic-action outcomes in the dataset. The dataset contains labeled CI failures but does not directly record the outcomes that would have been obtained by performing actions such as inspecting dependencies, comparing with a previous successful run, or inspecting server state. Consequently, the probabilities

$$P(o \mid H, E, a)$$

were estimated using action-specific proxies. These probabilities are used for simulation and should not be interpreted as directly observed action probabilities.

A second limitation is the size of the evaluation. Only 40 held-out cases were used. The resulting metrics therefore have substantial sampling uncertainty and should not be

Table 3: Comparison of the three diagnostic policies over 40 held-out evaluation cases.

Metric	EIG/cost	Cost-only	Baseline
Accuracy	55%	50%	25%
Macro Precision	65.23%	63.06%	6.25%
Macro Recall	55%	50%	25%
Human-review rate	35%	37.5%	0%
Average diagnostic cost	6.46	6.93	0
Average actions	4.68	5.23	0
Report precision	84.6%	80%	25%

treated as statistically conclusive evidence of production performance.

A third limitation is that the V1 model uses a relatively small hidden-state space. Infrastructure and External Service are represented conceptually but are not quantitatively evaluated because of data sparsity.

A fourth limitation is the reporting threshold. The 90% threshold was selected using intuition rather than being calibrated from historical decision outcomes. A future system could estimate an appropriate threshold from a larger collection of real diagnosis decisions and their consequences.

The system is designed to keep a human in the decision loop. When the agent cannot reach sufficient confidence after the useful available diagnostic actions have been exhausted, it escalates the case rather than forcing a root-cause report. This is particularly important because a confident but incorrect report can waste diagnostic effort and potentially mislead the human reviewer.

The probability record also provides an audit trail of the agent’s reasoning. It records evidence, hidden states, beliefs, actions, likelihoods, posterior probabilities, outcomes, decisions, and version information for an individual case.

The agent is therefore intended as a diagnostic aid rather than a replacement for human engineering judgment.

9 Conclusion and New Questions

This work presents a CI diagnosis agent that treats the root cause of a CI failure as a hidden state and uses Bayesian belief updates to reason about possible causes. Instead of following a fixed debugging sequence, the agent selects diagnostic actions using expected information gain relative to diagnostic cost. The result of each action is fed back into the belief engine, allowing the agent to update its beliefs and select another action.

The V1 evaluation compared EIG/cost, cost-only, and a historical-probability baseline over 40 held-out cases. EIG/cost achieved 55% accuracy, 65.23% macro precision, 55% macro recall, and 84.6% report precision. It also required fewer actions and lower average diagnostic cost than the cost-only policy.

At the same time, the failure analysis shows that the current system is limited by the quality of the simulated action outcome models. The agent can become confidently wrong, be diverted by misleading evidence, or remain below the reporting threshold even when moving towards the correct root cause.

These limitations lead to several questions for future versions of the system. Can action outcome probabilities be learned from real diagnostic-action logs instead of simulated proxies? Can the reporting threshold be calibrated from historical decisions and their consequences? Can additional evidence variables such as structured error messages, dependency changes, Docker state, and infrastructure signals be collected in a way that allows reliable likelihood estimation? Can the model distinguish Code failures from other root causes more reliably?

A larger dataset containing both the root cause and the results of diagnostic actions would allow these questions to be evaluated more directly.

Ethical Statement

The agent is designed to support diagnosis rather than replace human decision-making. Its probabilistic output can be incorrect, and the evaluation demonstrates several failure modes in which the agent either reports an incorrect root cause or cannot reach the required confidence. For this reason, the system includes human escalation when sufficient confidence is not achieved.

The main ethical concern in the current system is overconfidence. A high probability does not guarantee that the diagnosis is correct, particularly because the V1 action outcome probabilities are simulated rather than directly observed. The system should therefore not be treated as an autonomous authority for production incident decisions without further validation.

Contribution Statement

Problem selection: Panduru Grishm Siddharth.

Public discussions: Panduru Grishm Siddharth.

Agent design: Panduru Grishm Siddharth.

Software development: Panduru Grishm Siddharth.

Tests: Panduru Grishm Siddharth.

Citation checks: Panduru Grishm Siddharth.

Preprint sections: Panduru Grishm Siddharth.

PDF checks: Panduru Grishm Siddharth.

AI-Use Statement

AI tools were used during the project for coding and debugging assistance, mathematical explanations, research organization, editing and writing assistance, and data analysis. The

498 resulting code, analysis, decisions, and written material were
499 reviewed by the author.

500 **Code and Project Materials**

501 The code and project materials are available at:
502 <https://github.com/Proudprogamer/CI-Agent>

503 **References**